Guia de Programação: Tratamento de Erros (Módulo 09)

A lidar com `Exceptions` (erros). Usaremos a estrutura `try`, `except`, `else` e `finally` para que o nosso código seja "à prova de erros".

Desafio 1: A Calculadora de Divisão (Erros Matemáticos)

Neste código, aprendemos a capturar o erro de dividir por zero e o erro de digitar letras.

```
def aula_tratamento_erros():
    print("--- Desafio 1: Divisão ---")
    try:
        numerador = int(input("Digita o numerador: "))
        denominador = int(input("Digita o denominador: "))
        resultado = numerador / denominador
    except ValueError:
        print("Erro: Digita apenas números inteiros!")
    except ZeroDivisionError:
        print("Erro: Não podes dividir por zero.")
    except Exception as erro:
        print(f"Erro inesperado: {erro}")
    else:
        print(f"Sucesso! Resultado: {resultado}")
    finally:
        print("--- Fim da Divisão ---")
aula_tratamento_erros()
```

Desafio 2: Telemóvel e Duração (Tratando o Input)

Aqui usamos o `try` para converter uma duração (texto) em número para calcular o gasto de bateria.

```
class Celular:
    def __init__(self, marca, modelo):
        self.marca, self.modelo = marca, modelo
        self.bateria = 100

    def fazer_chamada(self, duracao):
```

```
        try:
            gasto = int(duracao) * 2
            if self.bateria >= gasto:
                self.bateria -= gasto
                print(f"Chamada de {duracao} min feita! Bateria: {self.bateria}%")
            else:
                print("Bateria insuficiente.")
        except ValueError:
            print("Erro: A duração deve ser um número inteiro!")

meu_celular = Celular("Samsung", "S24")
meu_celular.fazer_chamada("Dez") # Teste de erro
```

Desafio 3: Telemóvel e Custo (Tratando Tipos de Dados)

Neste exemplo, focamos no `TypeError`, que acontece quando tentamos fazer contas com tipos que não combinam.

```
class Celular:
    def __init__(self, marca, modelo):
        self.marca, self.modelo = marca, modelo
        self.bateria = 100

    def fazer_chamada(self, custo_bateria):
        print(f"\n--- Chamada no {self.modelo} ---")
        try:
            self.bateria -= custo_bateria
        except TypeError:
            print("ERRO: O custo da bateria deve ser um número!")
        else:
            print(f"Sucesso! Bateria atual: {self.bateria}%")
        finally:
            print("Sistema finalizado.")

meu_celular = Celular("Samsung", "S24")
meu_celular.fazer_chamada("muito") # Teste de erro
```

Desafio 4: Status de Bateria (Lógica + Erros)

O último desafio une tudo: tratamento de erro com `float` e condições `if/elif` para verificar a saúde da bateria.

```
class Celular:
    def __init__(self, marca, modelo):
        self.marca, self.modelo = marca, modelo

    def verificar_status(self):
        try:
            entrada = input(f"Bateria do {self.modelo}: ")
            nivel = float(entrada)
            if nivel < 0 or nivel > 100:
                print("Valor inválido! Digite de 0 a 100.")
            elif nivel < 15:
                print(f"⚠ Bateria em {nivel}%! Carregue já.")
            elif nivel > 85:
                print(f"✅ Bateria em {nivel}%. Carga excelente!")
            else:
                print(f"📱 Bateria em {nivel}%. Status normal.")
        except ValueError:
            print("Erro: Digite apenas números!")

cel = Celular("Samsung", "S24")
cel.verificar_status()
```